

AI SYSTEMS ENGINEERING • WEEK 4

Scoring Systems

One model, a number for every account. The architecture, the data, and the measurement.

Tonight

1. **Failure of the Week** — a resume scorer that learned the past
2. **Scoring systems** — a churn system: its architecture, its data, its measurement
3. **Design Studio** — design the delivery-time estimate
4. **Lab** — score every account, nightly and at the request

One to five stars

- In 2014 a large online retailer builds a system to score job applicants. A resume in, **one to five stars** out, the way shoppers rate products.
- About 500 models, one per job family and location; tens of thousands of terms.
- Trained on the resumes the company had received over the previous **ten years**.
- 2015: the scores for software jobs are lower for resumes containing the word *women's*, as in *women's chess club captain*, and for graduates of two women's colleges.

Source: Amazon's recruiting tool, as reported by Jeffrey Dastin, Reuters, 10 October 2018 · [reuters.com/article/us-amazon-com-jobs-automation-insight-idUSKCN1MK08G](https://www.reuters.com/article/us-amazon-com-jobs-automation-insight-idUSKCN1MK08G)

10 years
of hiring decisions were the dataset

Where did the dataset come from?

From a system built to hire people, not to specify a model.

The team wrote code: the features, the model, the pipeline. Every line of it was reviewed. None of it said what a good candidate was.

The labelled examples said that. Ten years of them, mostly men, because that is who had been hired. The model learned the dataset it was given.

Three facts about the data:

- The **label** was "was hired here": what the old process chose, not how anyone performed. The data could reproduce its preferences, not test them.
- The **rows** were the applicants the old process had kept. Nobody had a label for the ones it had turned away.
- The **fix** they tried, removing *women's* and the college names, edited the features. The signal was spread through the resumes, the edited models could not be shown clean, and the project was wound down.

What it cost

- the project was wound down by 2017; it never made a hiring decision on its own
- the story ran worldwide in 2018, and it is the example every regulator now cites
- the company said recruiters had looked at its recommendations

What a fix would have needed:

- a **dataset built on purpose**: where every row came from, a written labelling rule, a version
- a **metric per slice** before anything was built: the rate at which the score agrees with a reviewer, by gender, by school
- a **sample read by people regardless of score**, so the system could learn what the old process was missing

Each is a component. None is a model.

The same kind of bias has been found at other companies, and in other directions. For example, Google's pay-equity review found a bias in favour of women in software engineer pay (2019).

Where we are

#	STEP	SO FAR	TONIGHT
1	Scope & requirements	requirements as rates the three numbers	a churn system's requirements
2	Frame the AI task	the model's contract: inputs to outputs	a score from 0 to 1, per account
3	Metrics	rates with intervals the noise floor comparing versions judges	precision at k the holdout served = recomputed
4	Data	the golden set	getting production data the tables training sets where they are stored
5	Architecture	placement the queue the cascade	the batch scorer, and its orchestrator the online scorer
6	Deep dive		
7	Failure, scale & operations	the failure path	a failed run a stale table

A CHURN SYSTEM

Scoring Systems

A number for every account, used to decide who to call and what to offer.

A churn system

A video streaming service with 400,000 subscribers, each paying monthly.

One model. For each account it produces one number: *how likely is this account to cancel in the next 90 days?*

Two things use that number:

- **The retention team** phones the 5,000 most likely to leave, every week.
- **The cancellation page** decides whether to show a discount offer when a customer opens it.

THE NUMBERS	
accounts	400,000
the call list	5,000 a week, Monday 06:00
the cancellation page	about 2,000 visits a day
the truth	a cancellation, the day it happens; "stayed", 90 days after the as-of

The interview versions of this system: "design a churn prediction system"; "design fraud detection for card payments."

Three questions

For this system, as for every system from here on:

1. **What is the architecture?** Where the model runs, what surrounds it, what runs when it fails.
2. **How is the data handled?** Where the dataset comes from, how it is kept, what it specifies.
3. **How is it measured?** How you know the number is any good.

The model is **treated as given**: inputs to outputs, with a datasheet. Everything else is what we build.

SCORING SYSTEMS • SECTION 1

| The architecture

What shapes the design

- The call list is **read on Monday morning**. Nobody is waiting for it to be computed.
- The list is **read-only** to the people who use it. They look up a score; they never ask for one.
- Reading must be **fast**: a page of 250 names opens in under a second.
- The inputs, every account's history, **already exist** before anyone asks.
- There is a **whole weekend** to compute 400,000 scores.

The decision

Compute all the scores as a batch, at night.

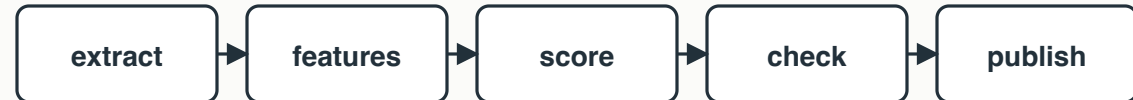
- every account, once a week, on Sunday night
- written to a table
- read on Monday morning

A nightly job

nightly.py · started by a timer at 03:00

```
rows    = extract(as_of)
X        = features(rows)
scores = score(X, weights)
check(scores, last_run)
publish(scores, run_id)
```

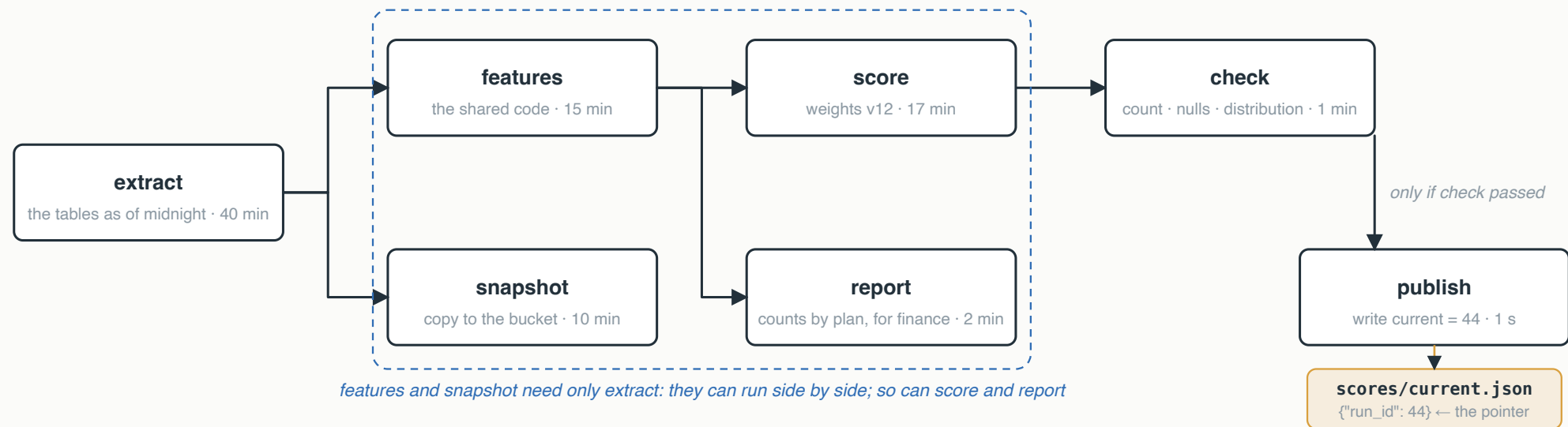
the same five steps



one after another, 03:00 to about 04:30

A batch job: five steps, in a row, once a night. Scores come out the end and are read on Monday.

Steps, and what each one needs



A step is a box. An arrow means "needs the output of." There are no loops. The orchestrator reads the arrows and works out the order itself.

Directed Acyclic Graphs (DAGs)

A job drawn as steps and arrows, where every arrow means "needs the output of" and no path leads back to where it started, is a **DAG**:

- **directed**: every arrow points one way, from a step to the steps that need it
- **acyclic**: following the arrows never brings you back to a step you have run
- **graph**: boxes and arrows

A batch job written this way can be handed to a program that runs it. That program is next.

What an orchestrator is

A program that runs DAGs for you. You give it the steps, the arrows, and a schedule. It:

- **starts** the job on the schedule
- **runs each step** as soon as the steps it needs have finished, and independent steps side by side
- **retries** a step that fails, on its own
- **stops** everything downstream when a step cannot be made to succeed, and tells someone
- **records** every run: each step's start, end and outcome, kept for as long as you want

What it does **not** do:

- it does not know what the steps compute; a step is a box it starts and waits for
- it does not hold the data; the steps read and write the tables
- it does not decide whether the output is any good; that is a step you write

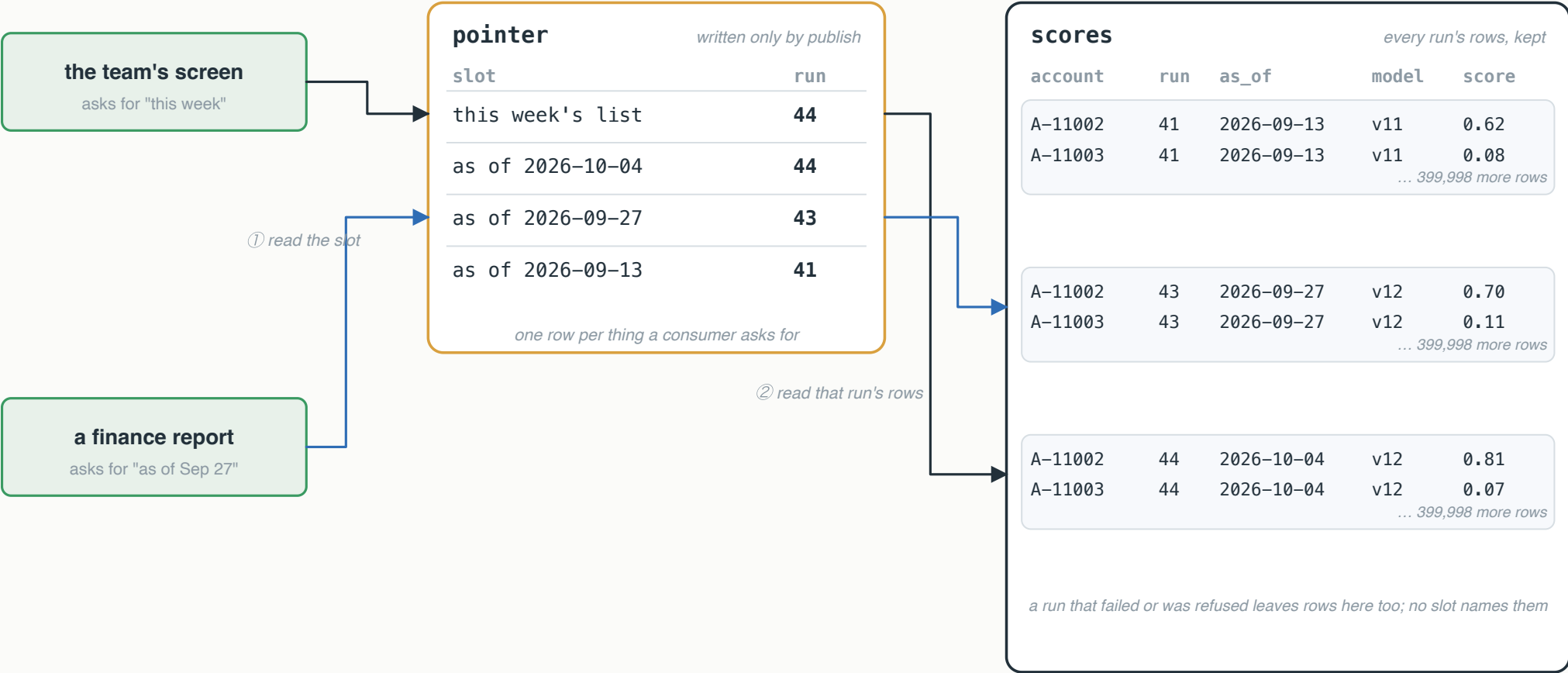
As of 2026, Airflow, Dagster, Prefect and Step Functions are orchestrators. The lab builds the mechanism from a calendar rule and a function.

Our job, on an orchestrator

- **the schedule**: Sunday 03:00
- **the DAG**: extract → features → score → check → publish, with snapshot and report beside them
- **the run record**: one row per step per run, on the right
- **publish** writes **the pointer**: a one-line record, `current run = 44`. The retention team's screen reads the run the pointer names, never the newest rows in the table. A run that never reaches publish is invisible to them.

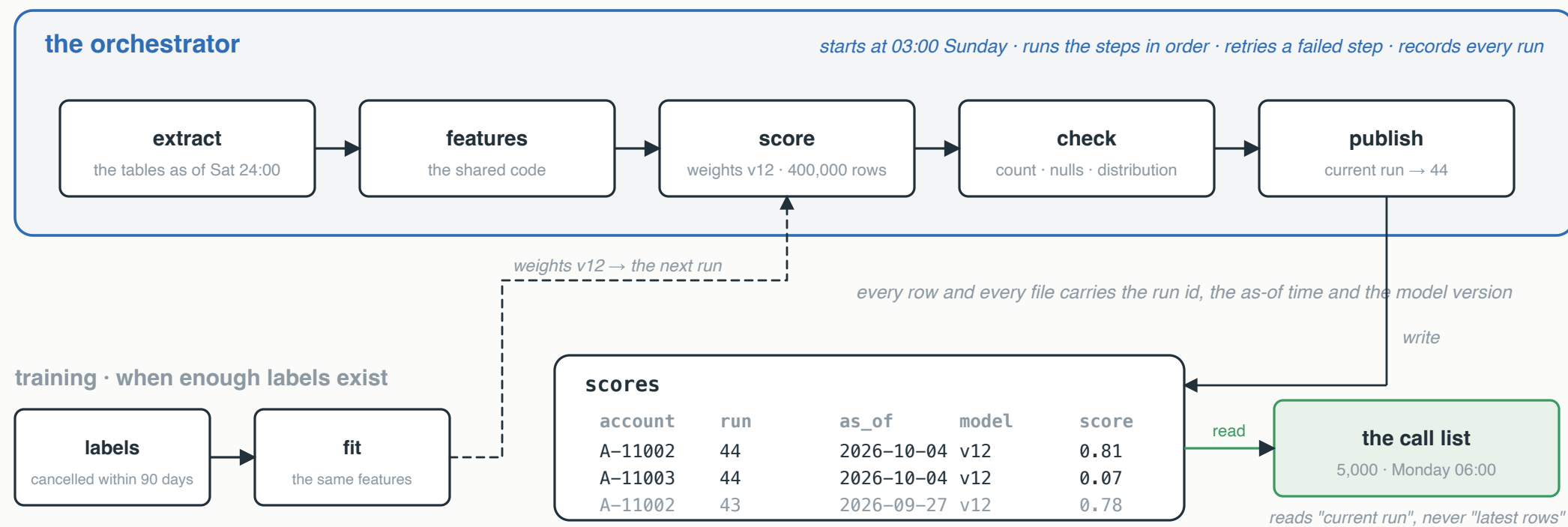
RUN	STEP	STARTED	ENDED	OUTCOME
44	extract	03:00	03:40	ok
44	features	03:40	03:55	ok
44	score	03:55	04:12	ok
44	check	04:12	04:13	ok
44	publish	04:13	04:13	ok

The pointer



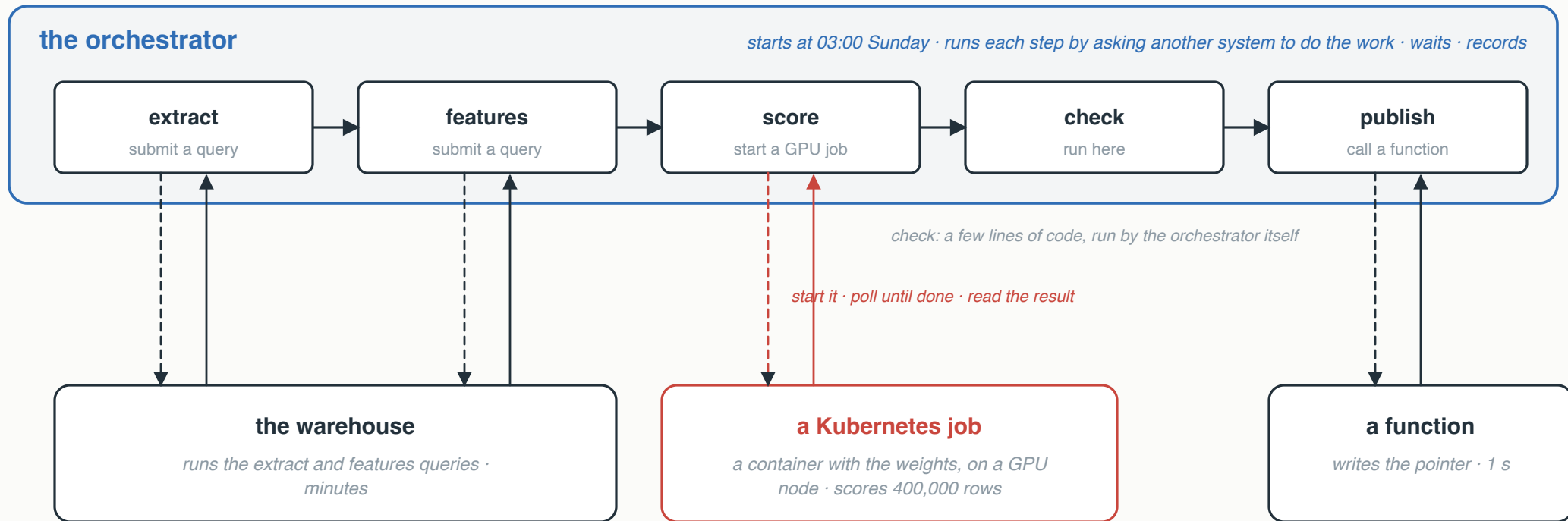
Two tables. The scores table keeps every run. The pointer says which run answers each question, and only publish writes it: a slot for this week, and a slot for each date.

The batch scorer



A batch scorer: five steps, run by an orchestrator, writing scores that are read later. Nobody is waiting; the schedule sets the freshness.

The same job, with the work done elsewhere



The same five steps. The orchestrator does none of the heavy work: each step is an operator that starts work elsewhere and waits for it. Score runs on a GPU; the orchestrator never needs one.

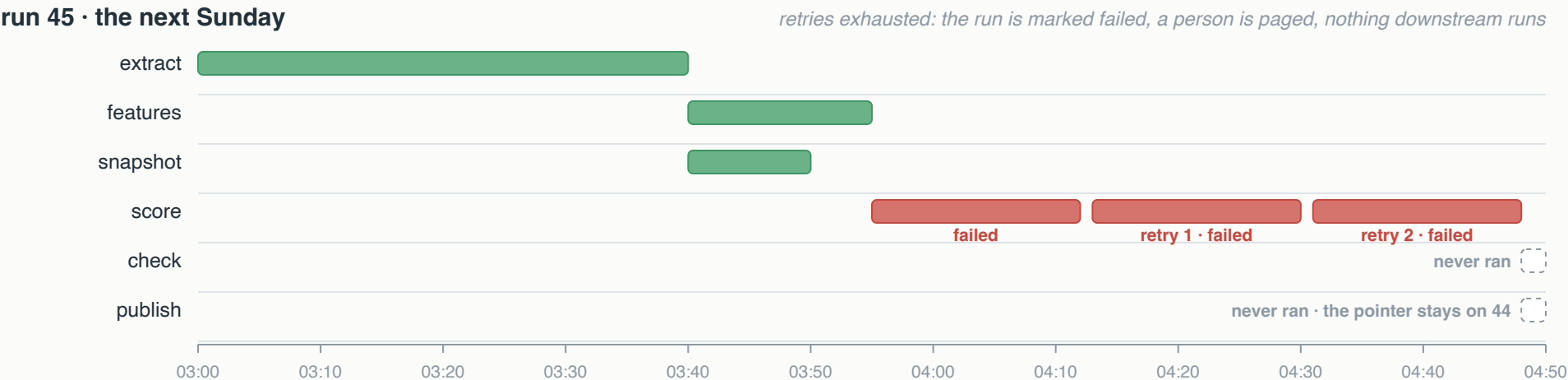
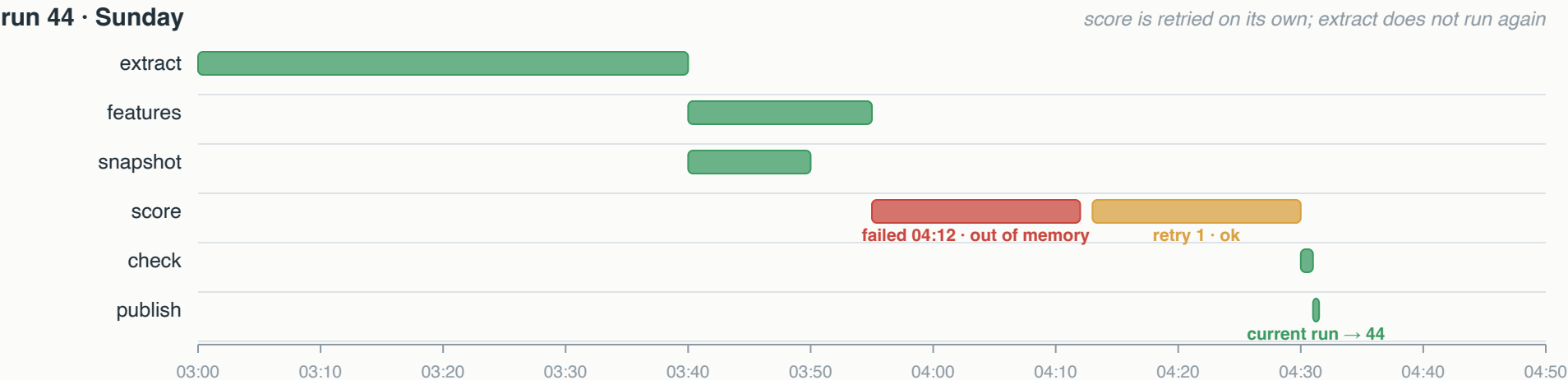
Choosing an orchestration mode

MODE	CHOOSE IT WHEN
The orchestrator does the work itself. Each step runs on the orchestrator's own machine.	every step fits on one ordinary machine. 400,000 rows through a logistic regression or a tree ensemble is minutes. One program, one machine, one log; a failed step fails where the orchestrator can see why.
Operators start the work elsewhere. Each step asks another system to do it and waits: a warehouse, a Kubernetes job, a function.	one step needs what the orchestrator's machine does not have: a GPU, a lot of memory, data too big to move. Each step pays for its own compute only while it runs. The orchestrator should never be the machine with the GPU.

SCORING SYSTEMS • SECTION 2

| Handling failures in a scoring system

When a step fails



A retried step, with a model in it

Run 44's score step died after writing **220,000** of 400,000 rows. The orchestrator ran it again.

OPTIONS	THE SCORES TABLE HOLDS
appends its rows	620,000 rows for run 44; 220,000 accounts scored twice
replaces its own run's rows, then writes all 400,000	400,000 rows
resumes : scores only the 180,000 not yet written	400,000 rows, and 220,000 fewer scores to compute

The two in green both work, and both need the same thing: **every row keyed by run id and account**, written so that a row is either there or not. Then a retry can rewrite everything or only what is missing.

Resuming is the common choice for a long step, and it is the better one when the step is a **language model**: the 220,000 rows already written are samples the run already paid for, and rescoreing them would replace them with different answers.

A run that is wrong, and "ok"

Run 45. Upstream, someone changed the refunds column from dollars to **cents**. Every step finished:

STEP	OUTCOME
extract	ok
features	ok
score	ok: 31% of accounts above the threshold
check	refused : last run 6%
publish	never ran

The orchestrator saw "ok" on the first three steps, because nothing crashed. The model scored every row faithfully, and every row was wrong.

Only the **check** step noticed, because it is the only step that compares this run with the last one. It refused to publish; the pointer stayed on 44; the team read last week's list, and a person was told.

What should we do in this case?

Rescoring the past

Weeks later. New weights, v12, are fitted.

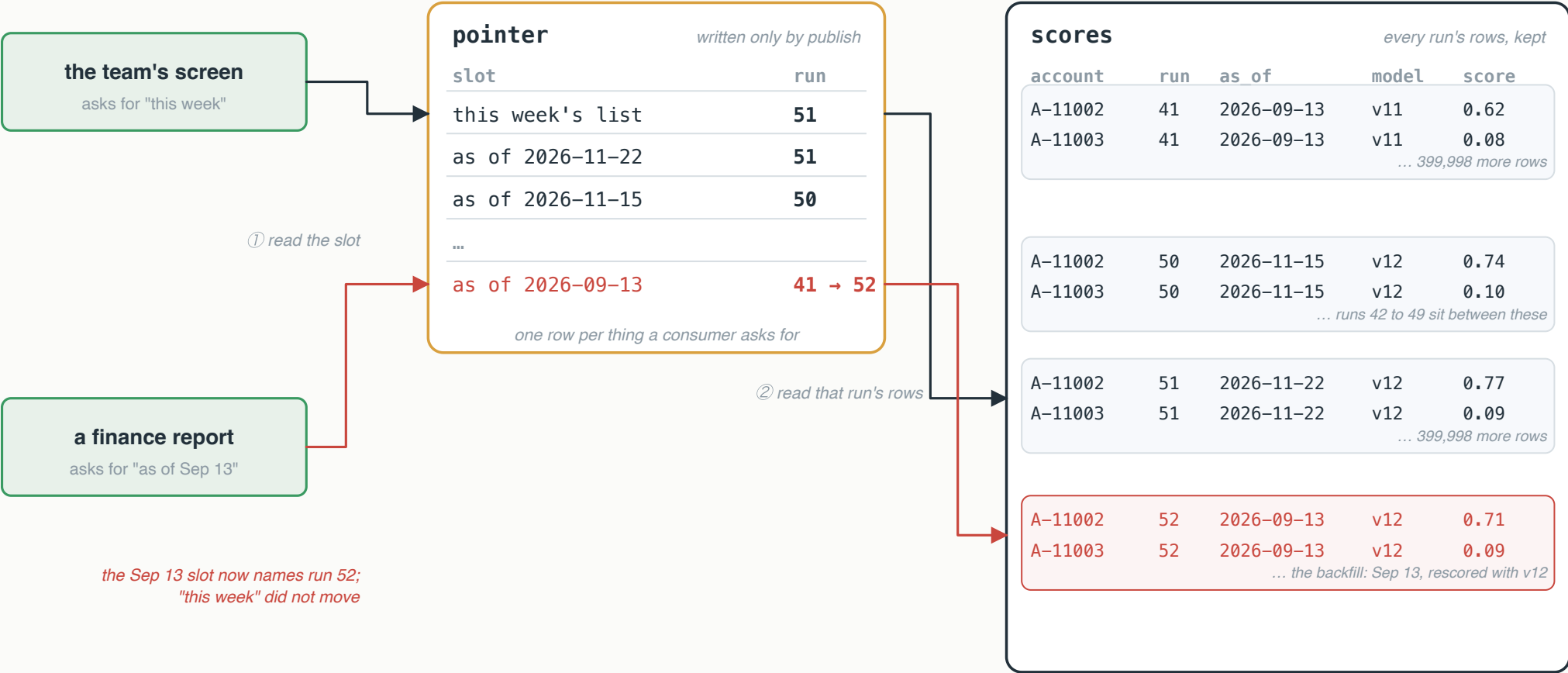
Someone asks: how would the September 13 list have looked with them?

The orchestrator runs the same DAG as a **new run** with an **old as-of**. The next run number is 52; the as-of is September 13, the date run 41 was for. The scores table now holds two scores for every account on that date:

ACCOUNT	RUN	AS_OF	MODEL	SCORE
A-11002	41	2026-09-13	v11	0.62
A-11002	52	2026-09-13	v12	0.71

A **run id** names one execution; every run gets the next number, whatever date it is for. The **as-of** says what the run was for. A backfill is a new execution for an old date, and the two fields together are what keep its rows apart from the original's.

The pointer, after a backfill

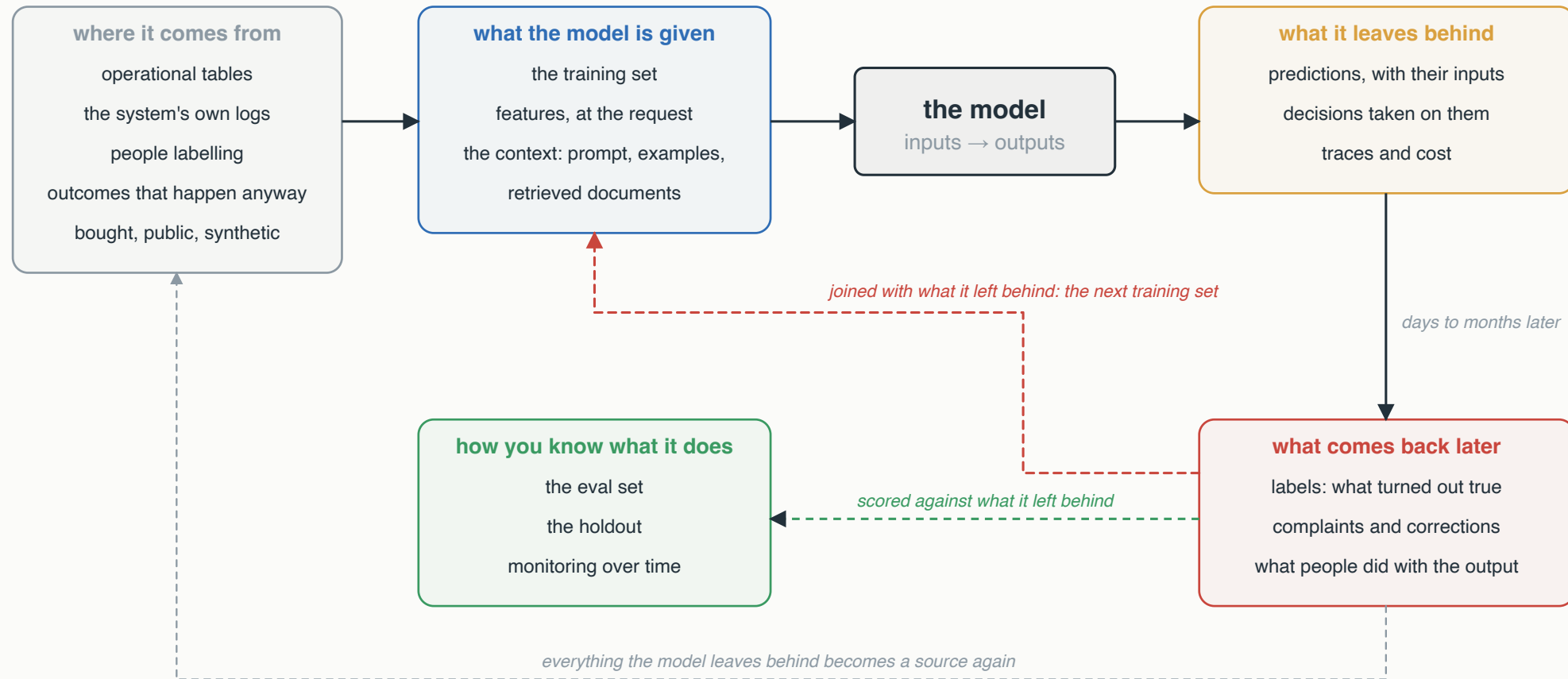


Weeks later, run 52 rescores September 13 with v12. Publish moves one slot: the one for that date.
Every other slot, and every other run's rows, are untouched.

SCORING SYSTEMS • SECTION 3

| The data

Where data lives in an AI system



Data in a production system: where it comes from, what the model is given, what it leaves behind, what comes back later, and how you know what it does. The model is treated as given; the rest is built.

What is different about data in production systems

ACADEMIC DATA	PRODUCTION DATA
downloaded; it existed before you did	does not exist yet ; you build it out of what a system left behind
one file, one owner	many systems, many owners , keys that do not match
the label is a column	the label is an event that happens later, somewhere else
fixed; train and test from the same pile	a stream ; the test set is next month
representative, or nobody asks	selected by the system itself : data about what it did, not what it did not do
no people in it	people in every row : consent, retention, who may look

Ways to get data

SOURCE	WHAT IT GIVES YOU	WHAT IT CANNOT GIVE YOU
your own system's log	exactly what was served, every time	anything from before the log existed
operational tables	the business as it is: accounts, orders, tickets	the moment of the decision; a label
outcomes that happen anyway	labels for free: cancelled, refunded, delivered	labels for what the system did not act on
people labelling	the label you actually want	scale; consistency without a written rule
a stronger model	labels at scale	a label better than the judge
bought, licensed, public	a start, a benchmark	your users
synthetic	coverage of rare cases; plumbing	the base rate; the surprises

A CHURN SYSTEM • THE DATA

| What data should we use for the churn model?

What the churn model needs

For every account, as of the day it is scored:

- **who they are** — the plan, the price, how long they have subscribed, how they pay
- **what they pay** — renewals, price changes, failed payments
- **what they watch** — what, when, how often, how recently, for how long
- **whether they asked for help** — tickets, complaints, what was done about them
- **what happened next** — cancelled or stayed, and when

Where does each line come from?

How does the churn system get data?

The sources, each owned by someone else:

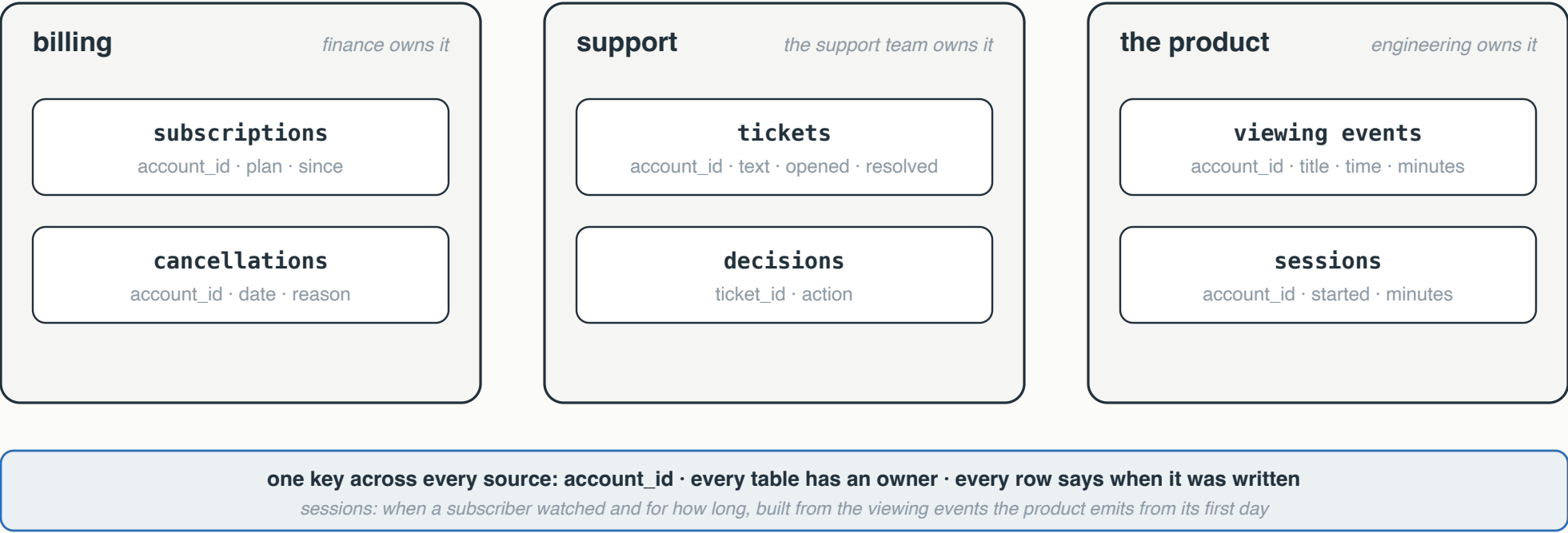
- **billing**, owned by finance: subscriptions, and cancellations with a date and a reason
- **support**, owned by the support team: tickets and what was done about them
- **the product**, owned by engineering: what each subscriber watched, and when

What the system has to put in place:

WHAT IS NEEDED	HOW
one key across every source	<code>account_id</code> , on every row of every source
viewing events	the product emits them from its first day: who watched what, when, for how long
an as-of on every row	so any date can be reconstructed
a record of every score	the scores table, with its run id and model version

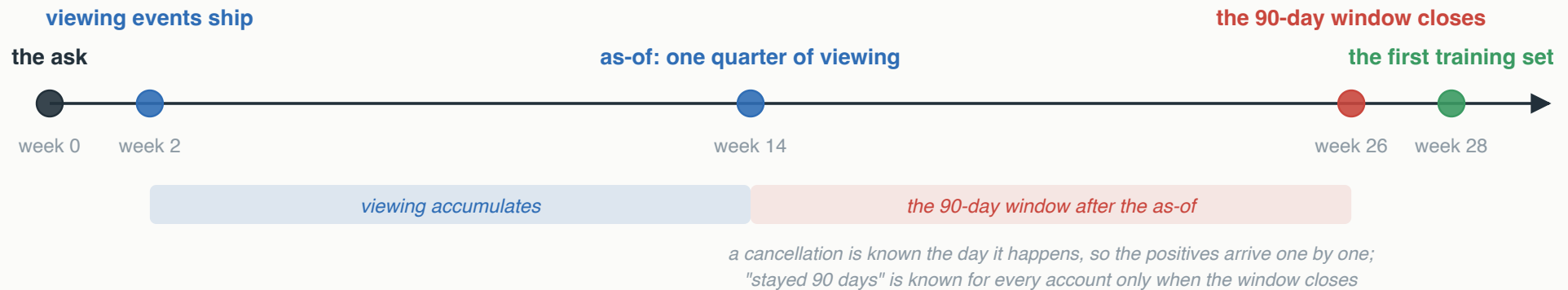
What has to exist before the first training set does?

Sources of data in the churn system



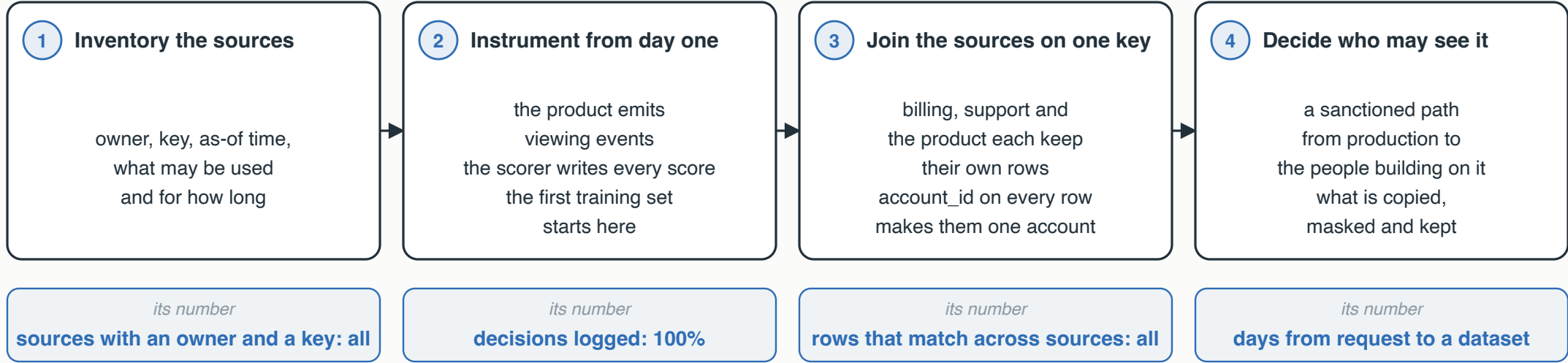
"Who is likely to cancel?" The sources, each owned by someone else, joined on one key the system was built to share.

When the data becomes available



The first training set is a date you can compute, set by the day the events start flowing. So the events ship first, before any model.

Building the data pipeline



Four mechanisms, each a box with a number. None of them is a model.

The dataset is the first system you build. The model is the second.

A year in, the retention manager asks: why was Account A-11002 on the September 13 list?

What we need to make this a reality

The scores table says run 41, v11, **0.62**.

To answer, and to turn that row into a training example, the system must still hold:

WHAT THE QUESTION NEEDS	WHERE IT LIVES
the score	the scores table: run 41, as-of September 13
the inputs the model saw	the features table, one row per account per as-of, never overwritten
the weights that scored them	v11, kept beside v12
whether the account cancelled	the labels table, 90 days later

The tables of an AI scoring system

inference

key · time · model_version
inputs as served · output
partition: day · append only

A-11002 · Sep 13 · v11 · 0.62

written by serving; never updated

features

entity_id · as_of
tenure · refunds_90d · ...
one row per entity per run

A-11002 · as_of Sep 13 · 9 · 1 · ...

written by a batch job; read by both paths

labels

key · label · arrived_at
cancelled · refunded · delivered
one row when the truth lands

A-11002 · cancelled · Oct 30

written by whatever learns the truth

Why was Account A-11002 on the September 13 list?

one row from each table, read by key and date, and the weights v11 from the object store: the production data, as it was written

object store

cheap · durable

weights · logs · frozen files

warehouse

joins · scans

the three tables

key-value store

milliseconds

what serving reads: the score by id

*The tables an AI scoring system writes as it runs, and where they live. They are never rewritten: a question about any past score is a read.
The inference table is the fastest-growing and most sensitive thing the system holds.*

The inference table

One row per score served, written by the score step and never updated. The nightly run's 400,000 rows are the batch scorer's inference table. Beside the account and the score, every row carries:

FIELD	WHY IT IS THERE
run id	which run produced this score; a retry replaces its own run's rows
as-of time	what the inputs were as of; the score's age when it is read
model version	which weights produced it; a rescoring with new weights is a new version

The pointer names a run id; the consumer reads the rows with that run id, and no others. Append only, partitioned by day: the fastest-growing and most sensitive table the system holds.

The features table

One row per account per as-of, written by the features step and never updated in place. The row is what the score step saw.

FIELD	WHY IT IS THERE
account_id · as_of	which account, and the date its inputs were read as of
run id	which run computed the row, with which version of the feature code
tenure · tickets_90d · refunds_90d · sessions_30d	the features themselves, by name, as the model received them

A new run writes new rows; last week's rows stay. That is what lets a past score be explained, and what lets its weights be refitted from the same inputs.

The labels table

One row when the truth lands, written by whatever learns it: billing, the day a cancellation happens; a job, the day an account's 90 days pass without one.

FIELD	WHY IT IS THERE
<code>account_id · as_of</code>	which account, and which score date the label answers
<code>label</code>	cancelled, or stayed
<code>arrived_at</code>	the day the truth was known; before it, this row did not exist

The positives arrive one at a time. "Stayed" arrives for everyone at once, when the window closes. A score has no label for 90 days, and a training set can only be cut at a date whose labels have all arrived.

Storage: the object store

What it is. Files, called objects, each under a name in a bucket. You write a whole object and read a whole object. There is no update in place and no query.

What it is for. Things written once and kept: frozen datasets, model weights, logs, fixtures. Any size, cheap per gigabyte, durable.

	THE OBJECT STORE
read	a whole object, by name: tens of milliseconds, plus its size
write	a whole object; a new version replaces, never edits
cost	cents per gigabyte a month
in the churn system	dataset v12, frozen to files with a hash · weights v11 and v12 · the raw event archive

Examples: S3, Google Cloud Storage, Azure Blob Storage.

Storage: the warehouse

What it is. Tables, stored by column, queried with SQL. One query scans billions of rows and joins across tables; the answer takes seconds to minutes.

What it is for. Building things from the whole history: the features job, the labels, the dataset query. Appending is cheap. Looking up one row is slow, and priced as a scan.

	THE WAREHOUSE
read	scans and joins over whole tables: seconds to minutes
write	append in batches; an update in place is expensive
cost	by bytes scanned and bytes stored
in the churn system	the inference, features and labels tables · the nightly features job · the query that becomes dataset v12

Examples: BigQuery, Snowflake, Redshift, Databricks.

Storage: the key-value store

What it is. One value under one key, in memory or close to it. Read a key in about a millisecond, write a key in about a millisecond, thousands of each a second. No scan and no join. It holds the keys that are read most often, as a cache in front of the slower stores.

What it is for. The things a request reads again and again while someone waits: the current score for an account, a session, the pointer to the current run, a lookup made a moment ago.

	THE KEY-VALUE STORE
read	one value, by key: about a millisecond
write	one value, by key; the newest wins
cost	by memory held: the most expensive per byte
in the churn system	the cancellation page's read: score by <code>account_id</code> · the pointer: the current run id

Examples: Redis, DynamoDB, Memcached.

Generating a training dataset



*A training set is a query over the production tables as of a date, frozen and numbered.
The tables do not change when it is made; the weights carry the number of the dataset they came from.*

Data as specification

FOR CODE	FOR THE DATASET
version control	a dataset version : the query, the as-of date, the hash
code review	data review : the source inventory, the join's match rate, the labelling rule
a unit test	a data test : schema, ranges, distribution against the last run, freshness
the build pins a commit	the build pins the dataset version the weights came from

Log what was served. Compute features once.

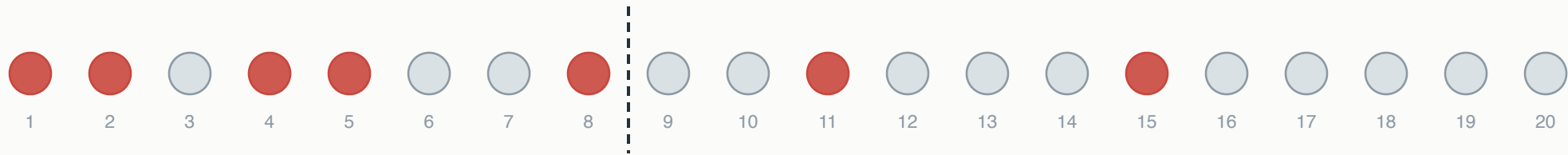
SCORING SYSTEMS • SECTION 4

| The measurement

A score, a line, four counts

accounts, in score order, highest first

● cancelled ○ stayed



the line: the top k, or a threshold

above the line, cancelled

TP = 5

above the line, stayed

FP = 3

below the line, cancelled

FN = 2

below the line, stayed

TN = 10

precision = TP / (TP + FP) = 5 / 8

of those above the line, how many cancelled

recall = TP / (TP + FN) = 5 / 7

of those who cancelled, how many were above the line

The model gives the order. The system draws the line. The labels, when they arrive, make four counts, and every eval of a score is a ratio of them.

The common evals for a score

EVAL	WHAT IT ASKS	COMPUTED AS
precision	of the accounts above the line, how many had the outcome	$TP / (TP + FP)$
recall	of the accounts with the outcome, how many were above the line	$TP / (TP + FN)$
precision at k · recall at k	the same two, with the line drawn at rank k	the counts above rank k
AUC	how well the order separates the two outcomes, over every line at once	the chance a random positive outranks a random negative
calibration	does a score of 0.8 have the outcome about 80% of the time	per score bin, mean outcome against mean score

Every one is a rate on labelled rows, with an interval, per slice.

TP, FP, FN: the four counts from the previous slide. AUC: area under the ROC curve, which plots recall against the false-positive rate as the line moves.

What they mean for the churn system

EVAL	FOR THE CHURN SYSTEM
precision at 5,000	of the 5,000 called on Monday, the share that would have cancelled
recall at 5,000	of everyone who cancels this quarter, the share that was among the 5,000 called
AUC	how well the order sorts cancellers from stayers, whatever the list length
calibration	whether a score of 0.8 cancels about 80% of the time

Each one per slice: new accounts, long-tenured, high-value. Each one with its interval.

What are the challenges with measuring this system?

- **The label arrives 90 days later.** A score made on September 13 cannot be judged until December. Every eval runs a quarter behind the score.
- **Calling the customer changes the outcome.** The list exists to be acted on. A called customer who stays is the point, not a wrong score, and the eval cannot tell a good score from a good call.
- **The first list has no history.** Nothing the system scored has a label yet.

How would we address these?

One solution: a holdout group

250 of the 5,000 are not called. Chosen at random, flagged at publish, in the inference table. The retention team's list is the other 4,750.

What it gets us

- an honest precision: of the 250, the share that cancelled, with nobody intervening
- recall too: below the line nobody is called, so those labels are honest anyway
- the value of the call itself: the 4,750 called against the 250 not

Before the first list: a backtest on past as-of dates.

What it costs. 250 customers a week not called: about 5% of the list.

What it can see. A precision of 20% on 250 rows carries an interval of about **15 to 25**. One Monday cannot tell 20% from 23%; a quarter of Mondays can.

$$\text{half-width} \approx 1.96 \sqrt{\frac{p(1-p)}{n}} = 1.96 \sqrt{\frac{0.2 \times 0.8}{250}} \approx 0.05$$

p: the measured rate · n: holdout rows with a label.

The pipeline's own measures

MEASURE	WHAT IT ASKS	REQUIREMENT
consistency	does a stored score equal the score recomputed from its feature row and its weights	exactly, on every sampled row
freshness	how old is the run the call list is reading	under 8 days
completeness	how many accounts got a score in the run	over 99.5%
on time	was the list published by Monday 06:00	50 of 52 weeks
label latency	how long from a score to its truth	90 days, by design

None of these needs a label; they are measured the morning after the run. The score is deterministic, so a stored score that differs from its recomputation is a bug, not a rate.

Implementing the measurement

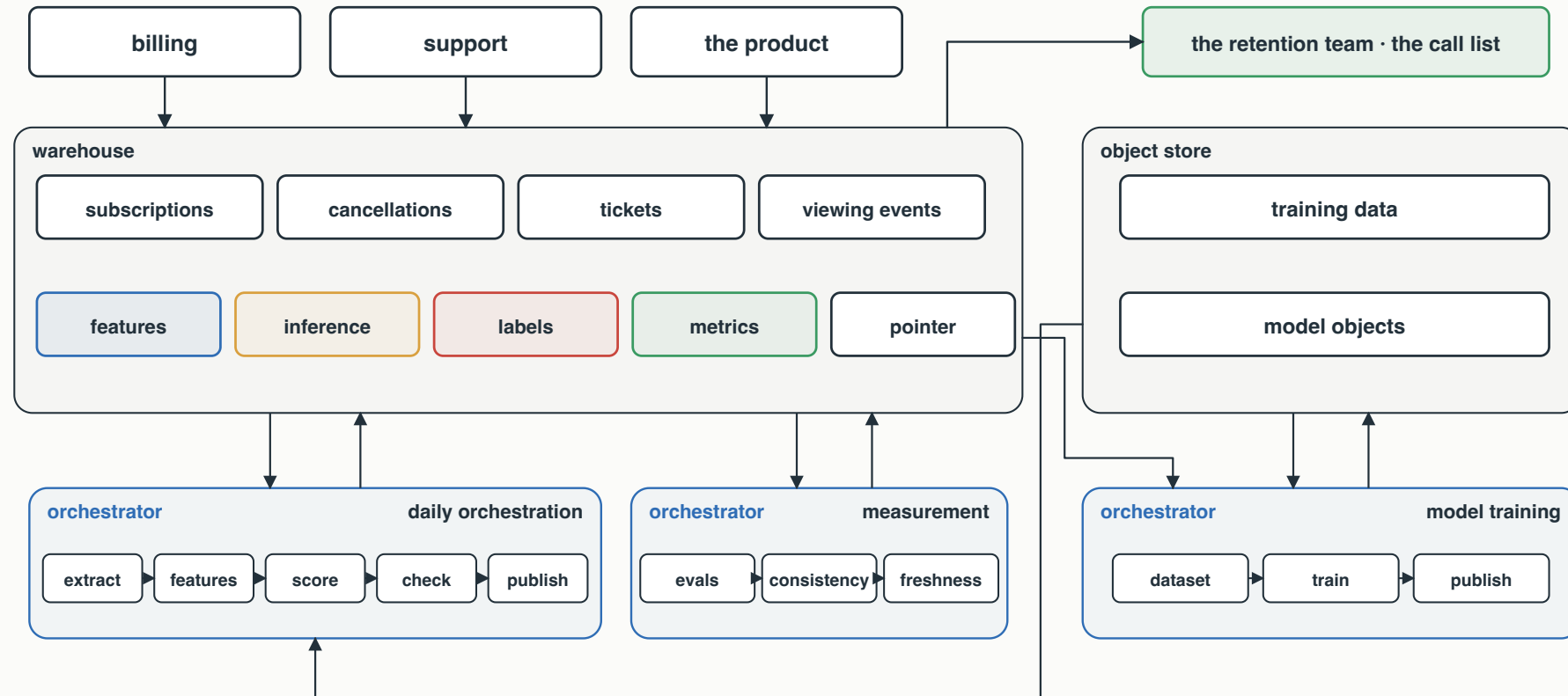


The measurement as components: jobs on their own clocks, one metrics table, one report. The holdout is a flag set at publish; nothing here reads a notebook.

The measurement so far

SYSTEM	WHAT IS MEASURED	THE INSTRUMENT
any model call	a rate on a slice, with its interval; the noise floor	the golden set; repeated runs
a change to a system	helped · hurt · cannot tell, per slice	paired differences
a judge, human or model	agreement with a reference	the four counts; κ
a scoring system	precision and recall at k, AUC, calibration, on a holdout; stored = recomputed; freshness, completeness	the inference table joined with the labels, 90 days later

The whole system



SCORING SYSTEMS • SECTION 5

| Adding online scoring

The cancellation page

The nightly list is done. Now the second consumer: a customer opens the cancellation page, and the app has **100 ms** to decide whether to show a discount offer.

- last night's score for this account exists, and is a lookup
- but the strongest signal is **what is happening now**: the customer is on this page, after this session
- that signal did not exist at 03:00

What was known last night is looked up. What is only known now is computed now.

A CHURN SYSTEM • ONLINE SCORING

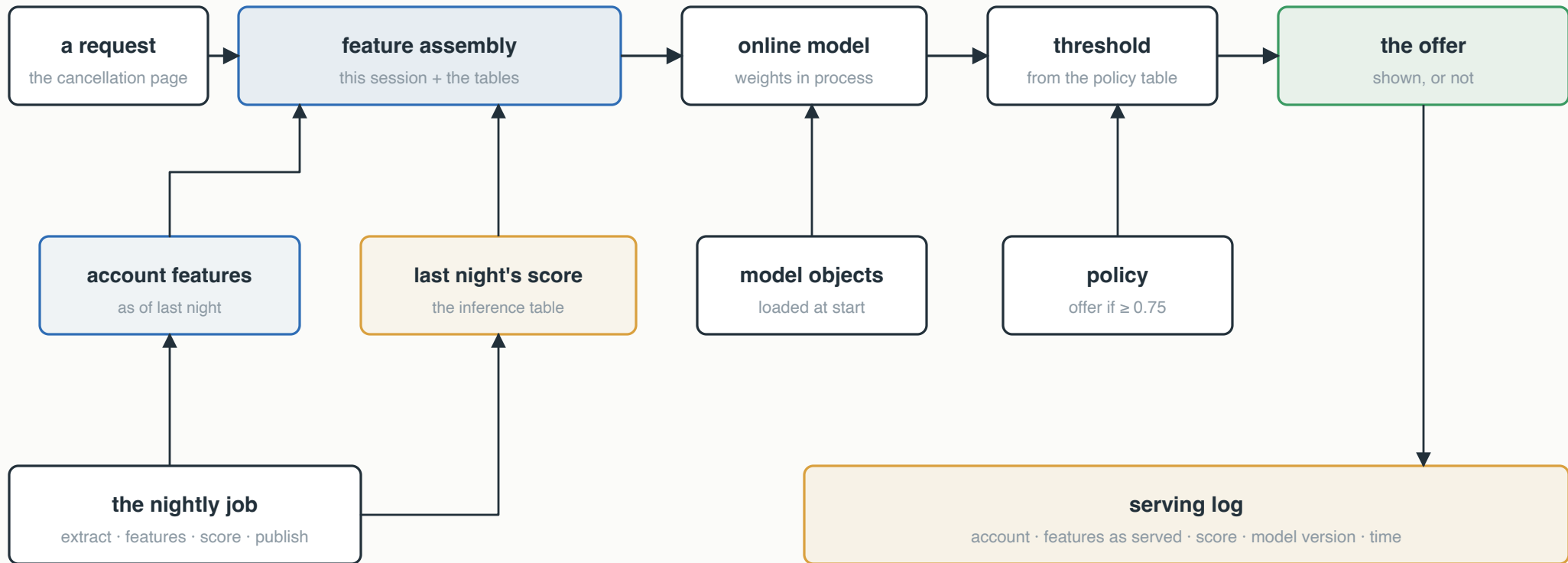
| The page has 100 ms. How do we want to score it?

What features do we want to add?

FEATURE	WHERE IT COMES FROM	WHEN IT IS KNOWN
last night's score	the inference table, by account	03:00
the account features	the features table: tenure, tickets, refunds, sessions	03:00
this session	the request: minutes watched, titles abandoned, device	now
this page	the request: the customer is on the cancellation page, and how they got here	now

A second model: last night's score and the account's features, plus what only the request knows.

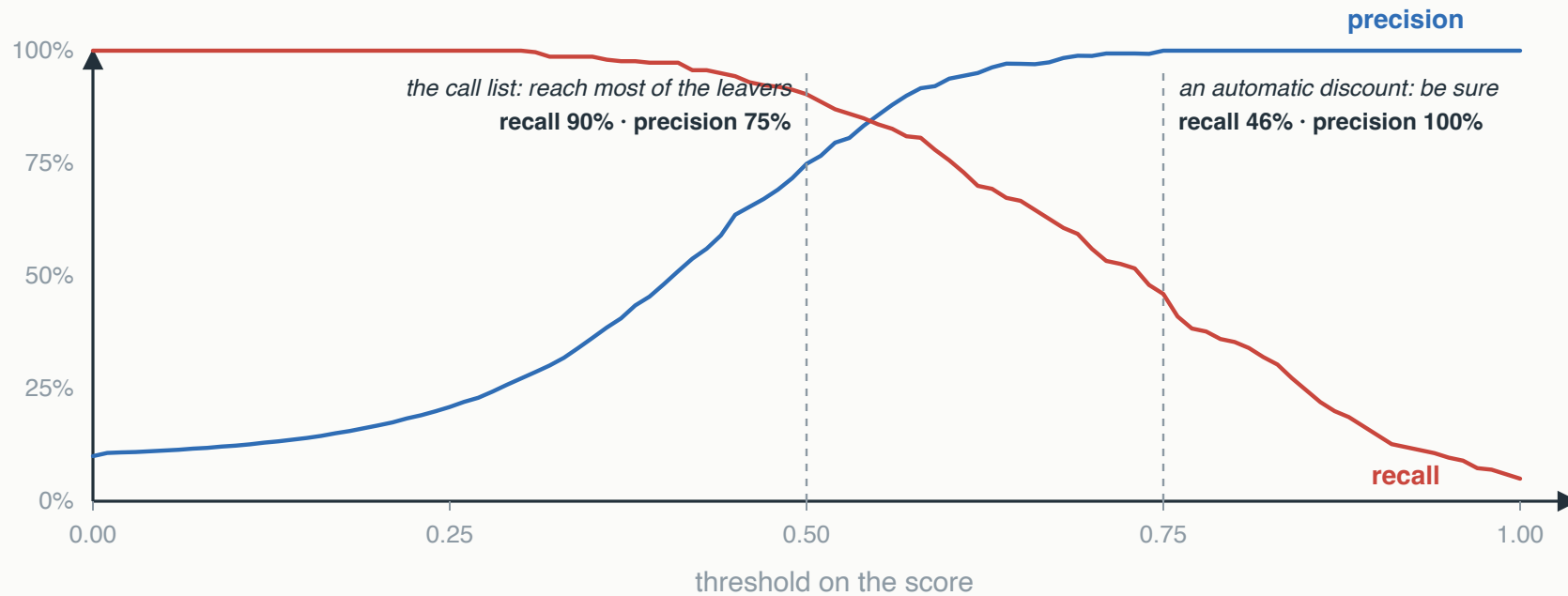
The online scorer



The online scorer's parts

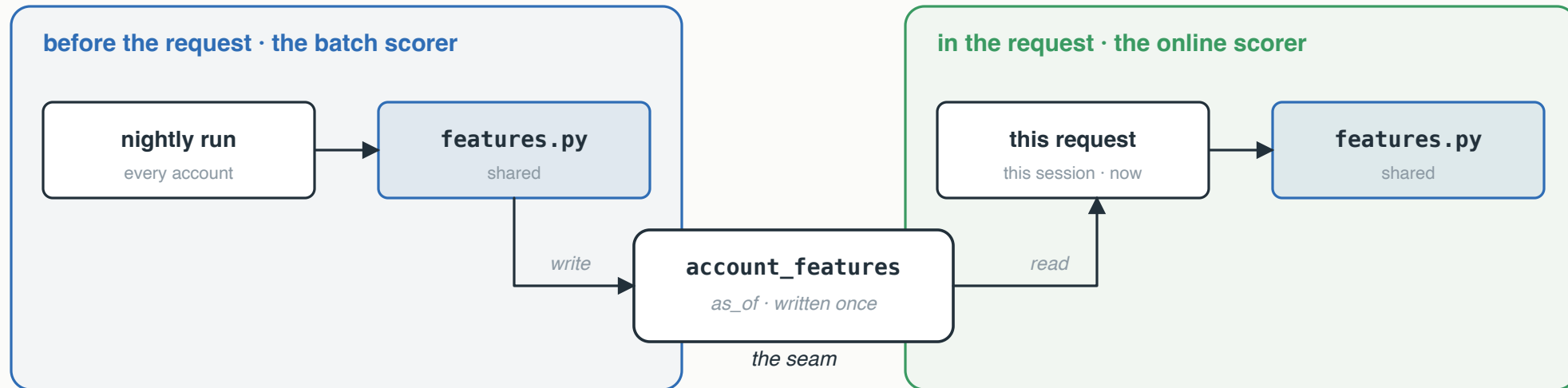
PART	WHAT IT DOES	WHAT THE MODEL CHANGES
feature assembly	joins what the request brings with the account features and last night's score	the tables' as-of time is part of the input
the model in process	a second model's objects, loaded at start; one inference per request, in memory	the cold start is the model load
the threshold	turns a score into a decision	a requirement in a table , not a parameter of the model
the serving log	the feature vector as served, the score, the model version, the time	the training set is what was served

The threshold



The threshold trades recall for precision. It is a requirement, written in a table, chosen by what the score decides. The model does not know it.

Where the two meet



*What can be known before the request is computed before it; what cannot is computed at it.
One feature code, one table between; the age of the table is a number the online scorer reports.*

What online scoring added

	NIGHTLY, FOR THE LIST	AT THE REQUEST, FOR THE PAGE
when the model runs	before anyone asks, on a schedule	when the customer arrives
what it sees	everything as of a cutoff	the request, plus the table as of last night
the dial	how often the job runs	the threshold
what is measured	precision at k on a holdout; freshness of the run	latency at the 99th percentile, under 100 ms; recall and precision at the threshold, per slice; a holdout of visitors shown no offer

When the online scorer fails

WHEN	WHAT RUNS	WHAT THE CUSTOMER SEES
the weights fail to load	no offer	the same page as for a low score
the feature table is stale	score anyway; flag the as-of on the log row	an offer decided on week-old history
the score takes longer than 100 ms	the page renders without waiting	nothing: the offer was never in the page's path
the threshold row is missing	no decision , and an alarm	a slower retention loop, never a wrong discount

Everything here degrades the offer, not the page.

Which question did they never ask?

- **How is the data handled?** Ten years of hiring decisions, taken from a system built to hire people: no inventory, no labelling rule, no version, and rows only for the applicants the old process had kept
- **How is it measured?** No metric per slice before the build: the rate at which the score agreed with a reviewer, by gender and by school, would have shown it in a week
- **What is the architecture** was the only question answered, and it was answered well

What a fix would have looked like:

- the dataset as a component: an inventory of where every row came from, a written labelling rule, a version
- the slice rates in the requirements table, with an owner
- a sample of applicants read by people regardless of score

DESIGN STUDIO

Design the delivery-time estimate

A food-delivery app shows a delivery time on every restaurant and again at checkout. A model produces it. Design the scoring system behind it: the architecture, the data, the measurement.

The full prompt is on the course site.

LAB • [GITHUB.COM/AISE-STTHOMAS/SCORE-EVERY-ACCOUNT](https://github.com/AISE-STTHOMAS/SCORE-EVERY-ACCOUNT)

Score every account, nightly

One weights file, two placements: a nightly batch run into a bucket with a run record, and the same score behind a URL. Fit it, join it point in time, run it, schedule it.

The instructions are in the repository's LAB.md.

Before you go

- **Reading for next week:** Lewis et al., *Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks* (2020), §1–3 · Manning, Raghavan & Schütze, *Introduction to Information Retrieval*, Ch. 1 and Ch. 6 §2 · Zheng et al., *Judging LLM-as-a-Judge* (2023), §1–3
 - **Project 1** is due; the harness you built scores the next project's gate. **Project 2** is assigned: the specification is in its repository.
 - Next week: retrieval-grounded assistants.
- ◆ *Before you leave: name one decision in a system you know that could not be reproduced from what the system wrote down.*